A Framework for Mesh-Network Simulation with CRDTs: An Analysis

Gabriela Rodrigues da Silva

Mesh networks allow replicas to communicate through intermediaries when direct connections are unavailable, which can improve the partition tolerance of distributed systems. This paper presents a C# framework for simulating operation-based CRDTs over mesh networks, with a modular network layer, causal delivery through version vectors and message buffering, and a visualisation tool for editing and observing networks. Flooding, distance-vector and link-state routing were compared across five network topologies in terms of completion time and number of messages sent. Flooding consistently sent the most messages and scaled poorly, distance-vector had the longest execution time, and link-state sent the fewest messages in all topologies except the complete graph. The results support the advantage of specialised routing algorithms over basic flooding and show that efficient message delivery is achievable in sparsely connected networks.

I. Introduction

Many distributed systems assume that replicas can communicate directly. When a direct connection becomes unavailable, messages may fail to reach their destination, compromising consistency [14]. Indirect paths may nevertheless exist, and forwarding messages through intermediary replicas could increase partition tolerance [25].

Mesh networking addresses similar problems. It describes infrastructures in which nodes connect directly, dynamically and non-hierarchically, and cooperate to route data efficiently [1].

Flooding is a routing technique in which a node forwards copies of an incoming packet to every outgoing link except the one it arrived on. Since it requires no pre-configured routing table, it is an important strategy for discovering available paths [26].

In contrast, distance-vector routing is a method in which each node maintains information about the distance to each destination and the next hop to reach it. Routers periodically exchange routing information with their neighbours and use metrics such as hop count or, in some protocols, link cost to determine the best route. Changes in network topology must be propagated periodically [10].

Link-state routing protocols instead store a graph representation of the network, from which each node independently calculates the best next hop to every destination. Whereas in distance-vector routing each node shares its routing table with its neighbours, in link-state routing it shares its list of available connections [11].

Many operation-based CRDTs [4] introduce the additional requirement of causal delivery: if a message M1 is sent after a set of operations has been observed, any node should receive M1 only after it has seen all operations in that set.

This project evaluates the effects of these three routing approaches on operation delivery latency and communication overhead across different topologies, while ensuring causal delivery.

II. Background

A. Conflict Free Replicated Data Types (CRDTs)

The CAP theorem is a fundamental principle in distributed systems, stating that a system cannot simultaneously guarantee consistency, availability, and partition tolerance. Consequently, trade-offs between these properties are necessary. In systems where availability is prioritised, eventual consistency is often adopted as an alternative to strong consistency. Eventual consistency guarantees that, in the absence of further updates, all replicas will eventually converge to the same state [22].

Conflict-free Replicated Data Types (CRDTs) are abstract data type designed to ensure eventual consistency in distributed systems while maintaining availability. CRDTs satisfy specific mathematical properties that enable replicas to be updated independently, without coordination. They guarantee that any two replicas that have received the same set of updates will deterministically converge to the same state, regardless of the order in which those updates were applied. This allows CRDTs to resolve inconsistencies arising from concurrent updates without requiring centralised coordination [23].

State-based CRDTs propagate their entire state to other replicas, which merge the received state with their local state. To ensure eventual consistency regardless of the order, frequency, or pattern of state propagation, the merge operation must be deterministic, commutative, associative, and idempotent. Furthermore, the state is modelled as a join-semilattice, ensuring that it evolves monotonically as updates are applied and replicas merge. A disadvantage of this approach is that it frequently involves relaying large amounts of data [3].

In contrast, Operation-based CRDTs allow for smaller messages, and more concise state — particularly when causality metadata is provided by the communication middleware. However, they generally require reliable causal broadcast, associated with communication metadata [4].

B. Mesh Networking

In full-mesh topologies, every node is linked directly to every other node. Because this requires $O(n^2)$ links, it is practical only for small clusters. Although direct links minimise communication latency, the probability that at least one link fails increases with the number of links. Partial-mesh topologies bound the number of links per node and therefore scale better, but the absence of direct links between some node pairs requires routing [17]. Common partial-mesh topologies include:

- **Ring:** each node connects to two neighbors, giving a fixed degree of 2 and a diameter of $\lfloor n/2 \rfloor$ for bidirectional links [15], [9].
- **Torus (k-ary n-cube):** nodes are arranged in an n-dimensional grid with k nodes per dimension and wraparound links, giving a constant degree of $2n$ and a diameter of $n\lfloor k/2 \rfloor$ [7], [9].
- **Hypercube:** 2^d nodes with degree d and diameter $d = \log_2 N$, where N is the number of nodes [15], [9].
- **Fat tree:** a hierarchical tree whose link capacity increases toward the root, providing high bisection bandwidth and multiple paths between nodes [18], [2].

Decentralized peer-to-peer (P2P) networks are self-organising distributed systems that do not follow the client-server model. They offer flexibility, high availability, robustness, load balancing, quality of service, scalability, decentralised administration, and anonymity. These systems are implemented as overlay networks, that is, virtual networks of logical links built on top of an existing network, which facilitate services such as message routing [20]. These networks can be broadly classified as structured or unstructured. Structured networks organise peers and content according to predefined rules, often using a distributed hash table (DHT) to enable efficient content discovery. In contrast, unstructured networks allow peers to join with little or no prior knowledge of the network topology and typically rely on flooding or other search-based mechanisms to locate content [19].

The design of a dissemination protocol requires deciding what is propagated and how. Flooding is a fundamental mechanism in which every node forwards each received packet once to all of its neighbors. It is simple and robust, but it generates many redundant copies. Gossip routing reduces this overhead by forwarding each packet only with a given probability [5]. When links are unreliable, point-to-point routing based on routing tables becomes preferable, for example with distance-vector or link-state algorithms. However, these approaches cannot deliver messages across network partitions, a limitation addressed by more advanced protocols designed for intermittent connectivity [24].

A further challenge in distributed systems is failure detection, because in an asynchronous network it is generally impossible to distinguish a failed node from one that is merely slow or poorly connected [12], [6]. This is commonly addressed with the following techniques:

- **Heartbeats and timeouts:** nodes periodically send liveness messages, and a node is suspected to have failed when no message arrives within a fixed timeout. A short timeout detects failures quickly but increases false positives, while a long timeout does the opposite [21], [13].
- **Adaptive failure detectors:** the timeout is derived from the observed distribution of heartbeat arrival times, so that it adjusts to changing network conditions. The φ accrual detector is one example, outputting a continuous suspicion level instead of a binary decision [16].
- **Gossip-based detection:** nodes periodically exchange membership and heartbeat information with randomly chosen peers, so that failure information spreads without central coordination and no single observer decides alone [27].
- **Indirect probing:** before declaring a node failed, the observer asks other nodes to probe it, which reduces false positives caused by a faulty link between the observer and the target [8].

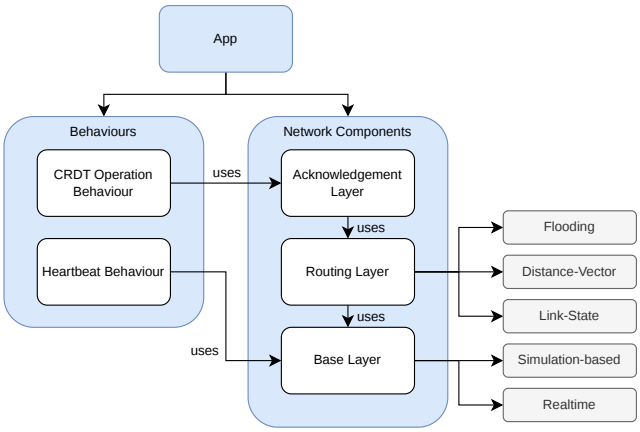


Fig. 1: Application Architecture

III. Solution

A. Overview

To avoid unpredictable effects, facilitate replication and speed up experiments, a network simulation system was developed alongside a small CRDT framework. The system is modular, so the networking component can be replaced without affecting the general routing behaviour. An experimental TCP-based implementation is also provided as a proof of concept for real-time use.

B. Technologies

The project is written in C#, taking advantage of its built-in libraries for parallel and concurrent programming and its object-oriented approach. Simulation uses SimSharp, a C# port of SimPy, which follows a discrete-event approach. Newtonsoft.Json is used for message serialisation.

C. System Architecture and Assumptions

Replicas are modelled as independent applications, although they are launched in parallel within the same process for simulation purposes. Each replica is associated with a set of background tasks and event handlers, encapsulated in two kinds of components: behaviours and network components (Figure 1). Behaviours control high-level CRDT operations and assume at-least-once delivery from the network components. The network components are responsible for tagging and propagating messages, and are where the routing algorithms are defined.

Although the framework can be customised for different sets of base network assumptions, all experiments were performed

under relatively limited guarantees at the lowest level of the network stack:

- Replicas execute concurrently and asynchronously.
- Without a dedicated heartbeat protocol, replicas do not know the status of their connections.
- Without a dedicated acknowledgement protocol, replicas do not know whether their messages were received.
- Replicas know each other's addresses.

A time-out-based heartbeat protocol and an acknowledgement protocol were nevertheless implemented on top of these assumptions, allowing replicas to detect interrupted or regained connections and to ensure delivery, respectively. Heartbeat messages are tagged with timestamps so that nodes can estimate communication latency.

D. Routing Algorithms

1) Flooding Algorithm

In the flooding algorithm, messages carry an additional field indicating their intended target, as well as an identification tag for de-duplication. Nodes forward every message not intended for themselves to all neighbours that the heartbeat protocol perceives as active. Each node keeps a set of recently seen tags and ignores the corresponding messages; these tags expire after a predetermined amount of clock time. Messages also have a maximum hop count. Together, these measures limit unnecessary message propagation.

2) Distance-Vector Algorithm

In the distance-vector algorithm, special algorithm messages are sent periodically to neighbours. They contain the distances a node has calculated so far to all other nodes. A distributed Bellman-Ford algorithm then determines the next node on the shortest path to each destination.

3) Link-State Algorithm

In the link-state algorithm, link-state advertisement (LSA) messages are flooded upon initialisation and whenever a network change is detected through heartbeat timeouts. Each LSA lists the node's currently active neighbours and the perceived distances to them, based on latency calculated from heartbeat timestamps. As LSAs eventually propagate to all nodes, each node obtains a full view of the network graph and uses Dijkstra's algorithm to determine the optimal paths.

E. Network Topologies

Five network topologies were tested:

- **Complete graph:** every node is connected to every other node. This makes routing algorithms superfluous, but is unrealistic for real-world scenarios.
- **Star topology:** a central node is connected to all other nodes.
- **Ring topology:** each node is connected to its two immediate neighbours, forming a cycle.
- **Erdős–Rényi random network:** each connection has a fixed probability of being present. The generation algorithm was adjusted to guarantee a fully connected network, as no routing algorithm can resolve partitions.
- **Barabási–Albert model:** generates random scale-free networks through preferential attachment, i.e. the probability of an edge existing depends on the number of edges the node already has.

F. Operation-based CRDT Implementations

Table I enumerates all implemented CRDTs. The ORSet was used as the basis for further testing owing to its greater complexity compared with the other implemented CRDTs.

TABLE I: Overview of the CRDT data types and their behavior

CRDT	Description
GCounter	Stores a per-replica increment counter. Its value is the sum of all increments. Operation messages contain only the value of the updated replica.
PNCounter	Stores two GCounters, one tracking per-replica increments and the other per-replica decrements. Its value is the difference between the total increments and decrements. Operation messages contain only the changed value.
GSet	A CRDT wrapper around a regular HashSet. Merging performs set-union. Items cannot be removed.
ORSet	Stores two sets of unique operation tags: one for every observed insert operation and one for every observed remove operation. An item is present if it has an insert tag that is not in the removed tag set. Merging performs the union of the corresponding sets. Insert messages contain only the added value and its tag; remove messages contain all tags observed for the value being removed.
LWWRegister	Stores a timestamp and value pair. Merging keeps the value with the most recent timestamp.
MVRegister	Stores a set of version vector-and-value pairs, retaining all values that occurred concurrently and discarding them once causally more recent values are observed.

G. Ensuring Causal Delivery

Causal delivery is ensured using version vectors and a message buffering layer. Each message is tagged with a monotonically increasing version vector. The buffering layer retains messages and passes them to the rest of the application only once their causal dependencies have been satisfied.

H. Visualisation Tool

A network editing and visualisation tool was built on top of the WPF framework (Figure 2). The main input page allows the initial network parameters to be configured. The tool then presents a graph view of the network, which can be edited by adding or removing nodes and connections. A real-time simulation can then be launched, displaying the different types of messages as they are sent.

IV. Evaluation

A. Experiments

To validate the framework, a simulation task was designed in which each replica performs a predefined number of insertions into an ORSet. Operations are issued at fixed time

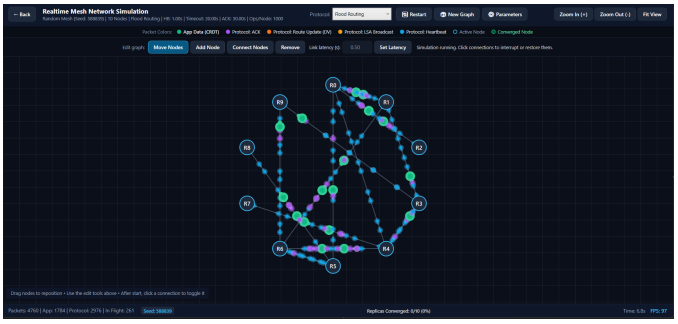


Fig. 2: Visualisation Tool Screenshot

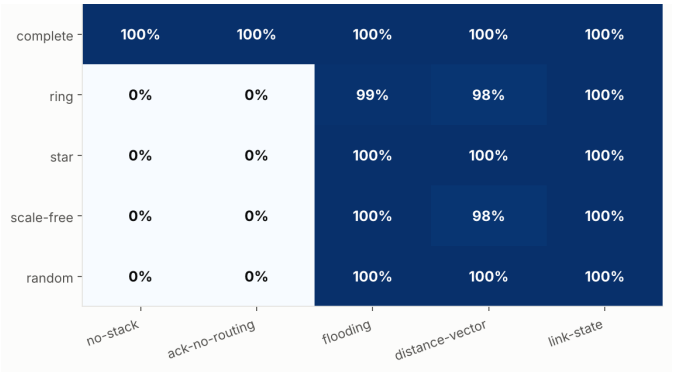


Fig. 3: Baseline Success Rate

intervals. Each replica repeatedly attempts to propagate operation messages to all other nodes using the routing algorithm until successful reception is confirmed by the acknowledgement protocol. Causal history is also tracked throughout the simulation.

The framework was evaluated under different network configurations, using 5, 8, 10, 12, 15, 18, and 20 replicas, with varying communication delays and algorithm parameters. For each configuration, the simulation's completion status, execution time, and number of messages sent before and after convergence were recorded. It must be noted that the number of messages includes heartbeat and acknowledgement messages, as they are essential to the correct functioning of the system as designed.

B. Results

In the baseline scenarios, the network has 10 replicas with link delays between 0.01 and 0.2 time units. Only the random and scale-free topologies use the full range; complete, ring and star use the minimum delay of 0.01 as a fixed delay. Each experiment runs for at most 200 time units. Each replica performs 5 operations, spaced 0.1 apart, and sends are spaced 0.4 apart. The heartbeat interval is 0.1, the timeout is 15, messages may travel up to 10 hops, the forgetting time is 60, the acknowledgement timeout is 5, and the distance-vector poll interval is 10. Topology-specific defaults are a random seed of 42, a centre node of 0 for the star, 2 edges per new node for the scale-free network, and an extra-edge probability of 0.15 for the random graph. The timeout adjustment can only raise values: the timeout is at least 2 × max delay + 3 × heartbeat (0.32 for the random and scale-free topologies, which is below 15), and the ack timeout is at least 2.5 × 9 × max delay (4.5 for random and scale-free, which is below 5). So for the baseline, the 15 and 5 defaults stay unchanged in every topology.

The resulting metrics are shown in Figure 7. The *no-stack* and *ack-no-routing* scenarios represent using no routing algorithm, without or with the additional heartbeat and acknowledgement protocol messages, respectively. They fail unless the network is a complete graph. Between the routing algorithms, the flooding algorithm consistently sends a greater number of messages, while the distance-vector algorithm consistently has a higher execution time. These algorithms only fail to terminate on limited occasions because the simulation is given a maximum time bound (Figure 3).

Figure 4 shows the increase in the number of messages sent as the number of replicas grows. The flooding algorithm showed the highest increase, particularly in networks with a higher frequency of connections, while the link-state algorithm showed the lowest number of messages in all cases except for the complete graph network. Figure 5 compares the number of messages of these two algorithms given random graphs with increasing numbers of edges and nodes.

Finally, Figure 6 shows the three parameters that most affect the number of messages for each algorithm compared with the baseline. Notably, the distance-vector algorithm was the most affected by algorithm timeouts, while the two others depended mostly on the number of operations and edges — predictably, a higher number of operations leads to a higher number of messages. The link-state algorithm varied with the random network seed, a parameter that would generally be expected to have little effect on the simulation. The *send-interval* metric affecting the flooding algorithm refers to the time between operation message emissions after a previous attempt to send the same message failed.

V. Discussion

A. Analysis of Results

The link-state (LS) algorithm performed well overall. In more highly connected networks, however, distance vector (DV) may be the better choice, since a higher number of edges increased the number of messages LS required before completion. This is consistent with how the two algorithms work. DV keeps sending routing information over time until it converges. LS may generate more data through its link-state advertisement (LSA) flooding mechanism, but it does so only at startup or when the network changes, which also makes updates propagate more quickly.

The randomisation seed was among the most influential parameters affecting the number of messages LS sent. Because the seed is a comparatively trivial source of variation, its prominence suggests that LS message overhead is less sensitive to substantive changes in the network than that of the other algorithms, whose message counts varied more with network parameters.

DV took the longest to run, which may be due to the initial propagation of distance vectors being delayed by timeouts.

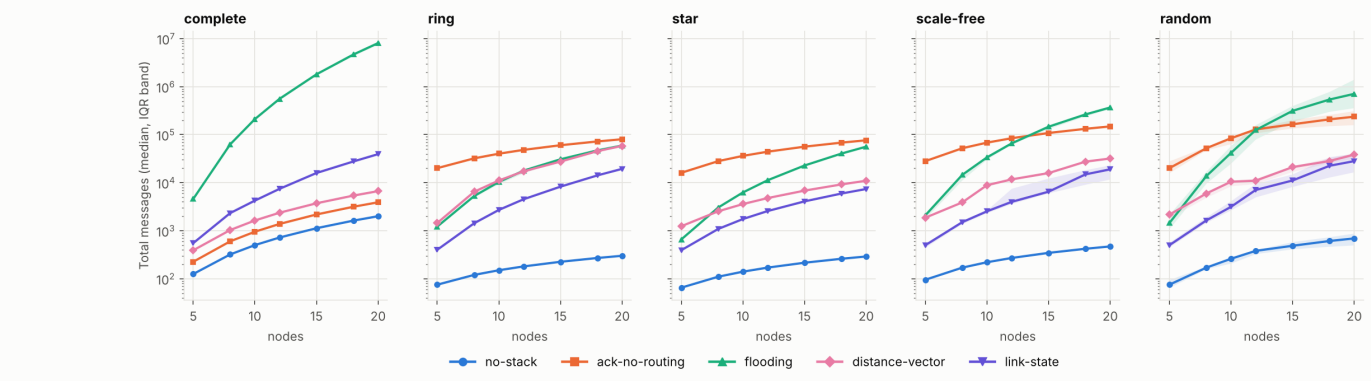


Fig. 4: Number of Messages per Number of Replicas

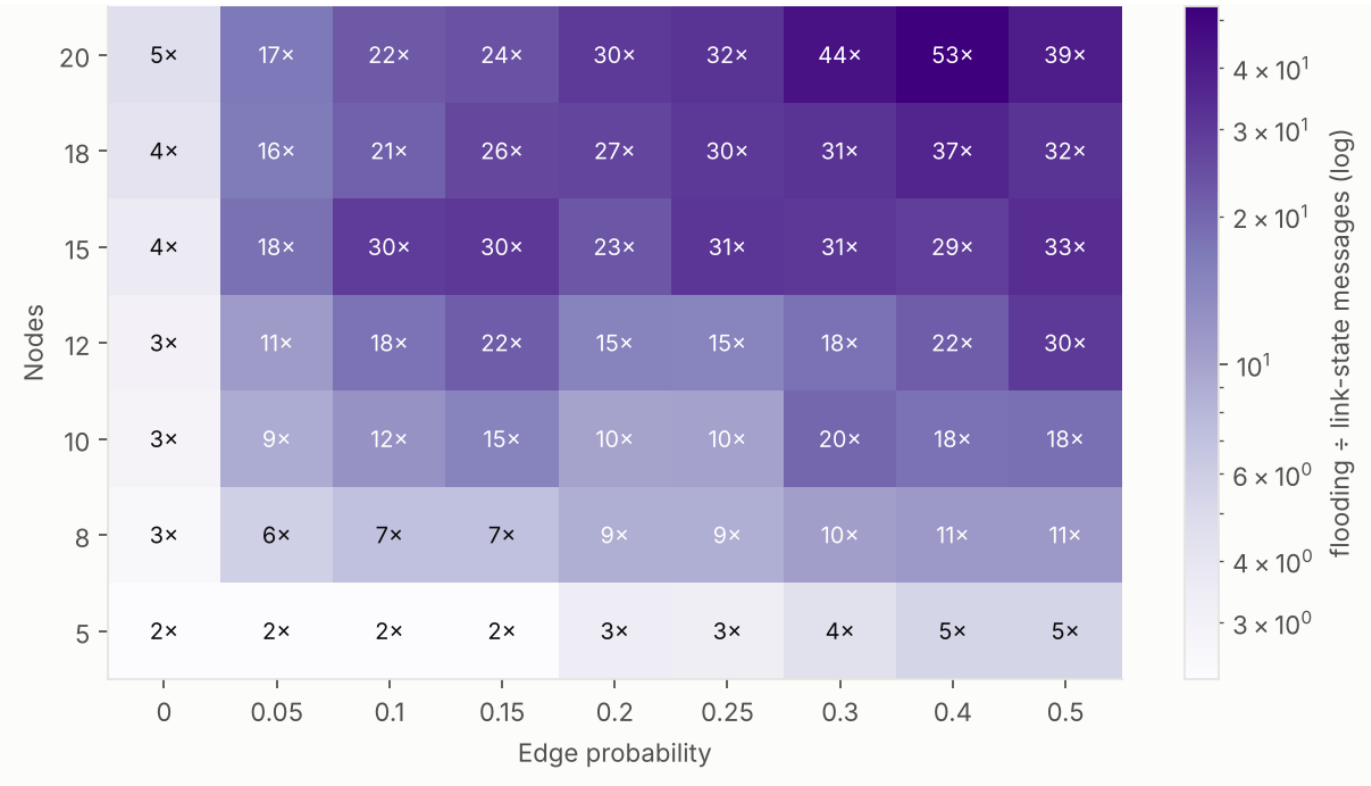


Fig. 5: Comparison of the Number of Messages Between the Flooding and Link-State Algorithms

This could be mitigated by adjusting the DV message interval dynamically, sending more messages when the network is estimated to be unconverged and fewer once it is estimated to have converged. The higher message count for DV may also follow from its longer execution time, since heartbeat messages must continue to be sent in the background.

The flooding algorithm scaled poorly. It placed a heavy computational load on the host machine during simulation and occasionally failed from timeouts in large networks, so it can cause failures that would not otherwise occur.

B. *Limitations*

- The causal delivery approach waits for every previous message implied by a version vector to be delivered before processing the corresponding message. In some scenarios, this is unnecessary, provided the final result dominates its causal predecessors.
- The discrete-event approach used in the main experiments assumes that computation time is negligible compared with network communication time, which may not always hold.
- JSON encoding may be too verbose; a more efficient serialisation format could reduce message size.
- The assumptions about replicas' mutual knowledge and the functioning of the system allowed the routing algorithms to be implemented in their simplest forms. Real-world protocols such as OSPF add further complexity.
- More CRDTs and test benchmarks could be developed.
- Although several network topologies were tested, dynamic aspects of real-world systems, such as node dis-

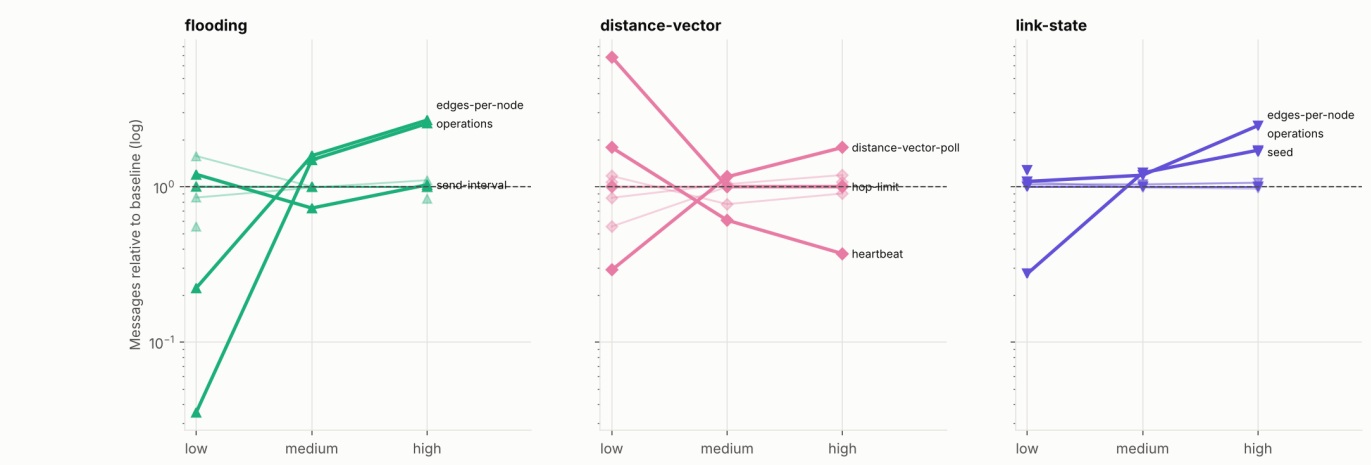


Fig. 6: Messages Per Variation in Simulation Parameters

connection and variable latency, were not fully explored.
- Simulations with a greater number of replicas were not attempted due to the limited computational power of the set-up, especially in the case of the flooding algorithm.
- The more linear execution of the simulation may hide concurrency or parallelism bugs.
- The visualisation tool is Windows-only.

VI. Conclusion

This project developed a C# CRDT framework, evaluated it through experimental simulations across different topologies, and produced a network visualisation tool. The results support the advantage of specialised routing algorithms over basic flooding, and show that efficient ways of sending messages can be found even for networks with sparse connections, while ensuring causal delivery via a message buffering mechanism.

References

[1] Ian F Akyildiz, Xudong Wang, and Weilin Wang. Wireless mesh networks: a survey. *Computer networks*, 47(4):445–487, 2005.

[2] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. In *Proc. ACM SIGCOMM*, pages 63–74, 2008.

[3] Paulo Sérgio Almeida. Approaches to conflict-free replicated data types. *ACM Computing Surveys*, 57(2):1–36, 2024.

[4] Carlos Baquero, Paulo Sérgio Almeida, and Ali Shoker. Pure operation-based replicated data types. *arXiv preprint arXiv:1710.04469*, 2017.

[5] Bastian Blywis, Mesut Günes, Felix Juraschek, Oliver Hahm, and Nicolai Schmittberger. A survey of flooding, gossip routing, and related schemes for wireless multi-hop networks. 2011.

[6] Tushar Deepak Chandra and Sam Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2):225–267, 1996.

[7] William J. Dally and Brian Towles. *Principles and Practices of Interconnection Networks*. Morgan Kaufmann, 2004.

[8] Abhinandan Das, Indranil Gupta, and Ashish Motivala. SWIM: Scalable weakly-consistent infection-style process group membership protocol. In *Proc. International Conference on Dependable Systems and Networks (DSN)*, pages 303–312, 2002.

[9] S. K. Dhall and S. Lakshmivarahan. Ring, torus and hypercube architectures/algorithms for parallel computing. *Parallel Computing*, 25(13–14):1877–1906, 1999.

[10] Peter L. Dordal. An introduction to computer networks. Open textbook, Loyola University Chicago, Department of Computer Science. Desktop edition, Release 2.0.11, Chapter 13.1: Distance-Vector Routing-Update Algorithm.

[11] Peter L. Dordal. An introduction to computer networks. Open textbook, Department of Computer Science, Loyola University Chicago. Desktop edition 2.0.11, Section 13.5: Link-State Routing-Update Algorithm.

[12] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

[13] Felix C. Freiling, Rachid Guerraoui, and Petr Kuznetsov. The failure detector abstraction. *ACM Computing Surveys*, 43(2), 2011.

[14] Seth Gilbert and Nancy Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *Acm Sigact News*, 33(2):51–59, 2002.

[15] Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar. *Introduction to Parallel Computing*. Addison-Wesley, 2 edition, 2003.

[16] Naohiro Hayashibara, Xavier Défago, Rami Yared, and Takuya Katayama. The φ accrual failure detector. In *Proc. IEEE Symposium on Reliable Distributed Systems (SRDS)*, pages 66–78, 2004.

[17] Huawei Technologies. Network fundamentals. In *Data Communications and Network Technologies*. Springer, 2022.

[18] Charles E. Leiserson. Fat-trees: Universal networks for hardware-efficient supercomputing. *IEEE Transactions on Computers*, C-34(10):892–901, 1985.

[19] Eng Keong Lua, Jon Crowcroft, Marcelo Pias, Ravi Sharma, and Steven Lim. A survey and comparison of peer-to-peer overlay network schemes. *IEEE Communications Surveys & Tutorials*, 7(2):72–93, 2005.

[20] Vassilis Moustakas, Hüseyin Akcan, Mema Roussopoulos, and Alex Delis. Alleviating the topology mismatch problem in distributed overlay networks: A survey. *Journal of Systems and Software*, 113:216–245, 2016.

[21] Marcia Pasin, Stéphane Fontaine, and Sara Bouchenak. Failure detection in large scale systems: a survey. In *IEEE Network Operations and Management Symposium Workshops*, 2008.

[22] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. *A comprehensive study of convergent and commutative replicated data types*. PhD thesis, Inria–Centre Paris-Rocquencourt; INRIA, 2011.

[23] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems*, pages 386–400. Springer, 2011.

[24] Thrasyvoulos Spyropoulos, Rao Naveed Bin Rais, Thierry Turletti, Katia Obraczka, and Athanasios Vasilakos. Routing for disruption tolerant networks: taxonomy and design. *Wireless networks*, 16(8):2349–2370, 2010.

[25] James PG Sterbenz, David Hutchison, Egemen K Çetinkaya, Abdul Jabbar, Justin P Rohrer, Marcus Schöller, and Paul Smith. Resilience and survivability in communication networks: Strategies, principles, and survey of disciplines. *Computer networks*, 54(8):1245–1265, 2010.

[26] Andrew S. Tanenbaum and David J. Wetherall. *Computer Networks*. Prentice Hall, Upper Saddle River, NJ, 5 edition, 2011. Section 5.2.3, "Flooding".

[27] Robbert van Renesse, Yaron Minsky, and Mark Hayden. A gossip-style failure detection service. In *Proc. IFIP Middleware*, pages 55–70, 1998.

APPENDIX A
GENAI USAGE

LLMs such as ChatGPT and Claude were consulted to find references, such as papers on specific topics and frameworks suited to a given purpose, and to confirm small pieces of information, such as brief definitions of terms and whether particular approaches had already been tried or were known to work. They were also used to generate the basis for the unit tests, which were then individually verified, and to identify potential bugs. The visualisation app was developed with AI guidance for page layouts. AI was also used for language assistance while writing this report.

APPENDIX B
BASELINE METRICS

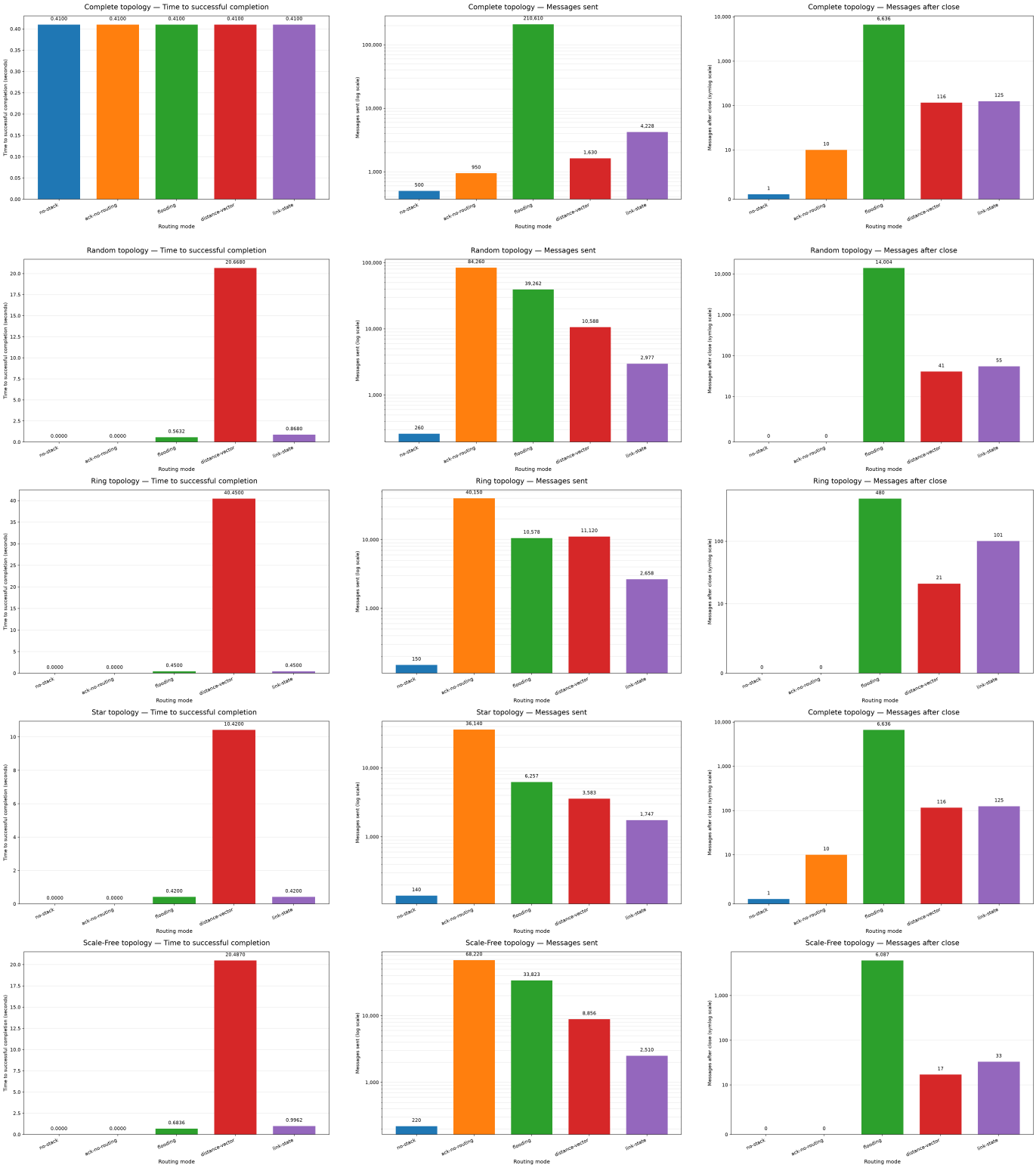


Fig. 7: Baseline metrics across every algorithm-topology combination.